

Project Work for Deep Learning:

Sexism Detection using RNNs and Transformers

Usama Shabbir Satti

Student ID: 0001158315

usama.shabbirsatti@studio.unibo.it

April 1, 2026

Abstract

This report details the experimental methodology, implementation, and evaluation of deep learning models for multi-class sexism detection in social media texts. The objective is to categorize sexist messages based on author intention into three distinct categories: Direct, Reported, and Judgemental, alongside a Non-sexist class. I implemented and evaluated Recurrent Neural Networks (Baseline and Stacked Bidirectional LSTMs) using GloVe word embeddings, as well as a state-of-the-art Transformer architecture (Twitter-roBERTa-base). Through robust evaluation across multiple random seeds, the Transformer model significantly outperformed the LSTM counterparts, achieving a Macro F1-score of 0.4807 on the test set. Extensive error analysis reveals persistent challenges with severe class imbalance, particularly in identifying the minority 'JUDGEMENTAL' class. I conclude with strategic recommendations for future performance improvements.

1 Introduction

Social media platforms are frequently exploited to disseminate harmful content, including sexist and misogynistic discourse. Automating the detection and classification of such content is a critical step toward creating safer online environments. The primary problem addressed in this project is the categorization of sexist messages according to the specific intention of the author.

This is framed as a multi-class text classification task featuring four distinct labels:

- **NON-SEXIST (-):** The tweet does not contain sexist content.
- **DIRECT:** The intention is to write a message that is sexist by itself or incites sexism. Example: "A woman needs love, to fill the fridge..."
- **REPORTED:** The intention is to report and share a sexist situation suffered by a woman or women. Example: "Today, one of my year 1 class pupils could not believe he'd lost a race against a girl."
- **JUDGEMENTAL:** The intention is to condemn or judge sexist situations or behaviours. Example: "As usual, the woman was the one quitting her job for the family's welfare..."

Given the semantic complexity and nuanced differences between 'Reported' and 'Judgemental' intentions, advanced Natural Language Processing (NLP) and Deep Learning architectures are

required. This report documents the complete pipeline from data preprocessing to model evaluation, covering two LSTM-based architectures and one Transformer-based architecture.

2 Corpus and Data Preprocessing

2.1 Dataset Description

We utilized a subsampled version of the EXIST dataset, split into training, validation, and test sets stored in JSON format. The dataset includes annotations by six different annotators, featuring demographic metadata such as gender (F, M) and age brackets (18-22, 23-45, 46+). Each entry contains the following fields:

- `id_EXIST`: Unique tweet identifier
- `lang`: Language of the tweet (en/es)
- `tweet`: The raw tweet text
- `number_annotators`: Number of annotators (always 6)
- `gender_annotators / age_annotators`: Annotator demographic metadata
- `labels_task1`: Binary sexism labels per annotator
- `labels_task2`: Intention labels per annotator (the target task)
- `labels_task3`: Fine-grained category labels per annotator

For this project, we exclusively focused on Task 2 (Sexist Intention) and restricted the corpus to English-language tweets (`lang == 'en'`). An example dataset record is shown below:

```
{ "id_EXIST": "203260",
  "lang": "en",
  "tweet": "ik when mandy says you look like a whore i look cute as FUCK",
  "labels_task2": ["DIRECT", "DIRECT", "REPORTED", "-", "JUDGEMENTAL",
    "REPORTED"],
  "split": "TRAIN_EN" }
```

2.2 Data Loading

The three JSON files (training, validation, test) were loaded using a custom function that reads the JSON with `orient='index'` formatting. The loading was implemented using pandas:

```
def loadDataFromDrive(file_path):
    return pd.DataFrame(pd.read_json(file_path, orient='index'))
```

After loading, the raw dataset shapes before any preprocessing were verified. The data pipeline then applied a unified `handle_data()` function which chains label aggregation, language filtering, and column pruning into a single reproducible preprocessing step.

2.3 Label Aggregation

Because six distinct annotators labeled each tweet, we applied a strict majority voting aggregation strategy. A clear majority was defined as a label receiving at least three votes and strictly outnumbering any competing label. This was implemented via the `maj_label2()` function:

```
def get_majority_label(row_labels):
    value_counts = pd.Series(row_labels).value_counts()
    max_count = value_counts.max()
    if max_count >= 3:
        candidates = value_counts[value_counts == max_count].index.tolist()
        if len(candidates) == 1:
            return candidates[0]
    return np.nan # No clear majority - will be dropped
```

Rows without a clear consensus were dropped from the dataset to ensure high-quality ground-truth labels. The final consolidated labels were mapped to integer IDs for network processing:

Label	Description	Integer ID
-	Non-Sexist	0
DIRECT	Directly sexist or inciting sexism	1
JUDGEMENTAL	Condemning/judging sexism	2
REPORTED	Reporting a sexist situation	3

2.4 Data Cleaning

Social media text is notoriously noisy. To construct a standardized vocabulary and reduce spurious feature learning, we implemented a rigorous text cleaning pipeline for the LSTM architectures. The `clean_text()` function performed the following operations in order:

1. URL Removal: Stripped all http links using regex `r'http\S+'`
2. Hashtag Removal: Removed `#hashtag` tokens using `r'#\w+'`
3. Mention Removal: Removed `@user` mentions using `r'@\w+'`
4. Emoji Filtering: Emojis were removed using the emoji library's `replace_emoji()` function
5. Character Stripping: Special characters and symbols filtered using `r'^A-Za-z0-9\s'`, preserving only alphanumeric strings
6. Empty Tweet Removal: Tweets reduced to empty strings after cleaning were dynamically filtered out

After cleaning, a combined Stemming and Lemmatization pipeline was applied using NLTK's `SnowballStemmer` and `WordNetLemmatizer` with POS-tag awareness:

```
def lem_stem(text: str):
    stemmed = stem_text(text) # SnowballStemmer
    lemmem = lem_text(stemmed) # WordNetLemmatizer with POS tags
    return lemmem
```

POS-tag aware lemmatization was implemented via a helper that maps Penn Treebank POS tags (J/V/N/R) to WordNet constants (ADJ/VERB/NOUN/ADV). This combined approach reduces vocabulary size and maps morphological variants to shared roots.

An analysis of the label distribution post-cleaning revealed a heavy skew toward the non-sexist class (-), indicating a severe class imbalance that would later impact model performance. The NON-SEXIST class constituted the vast majority of samples across all splits.

3 Text Encoding

To feed textual data into the Recurrent Neural Networks, we constructed an integer-based vocabulary mapping. A fixed vocabulary was derived from the training set using the `build_vocabulary()` function, which assigns a unique integer index to every unique token encountered.

The vocabulary construction reserved:

- Index 0 for the padding token <PAD>
- A dedicated index for the out-of-vocabulary token <UNK> (appended after vocabulary construction)

We utilized GloVe embeddings (glove-twitter-50), trained specifically on Twitter corpora, with an embedding dimension of 50. These were loaded via the gensim downloader API:

```
glove_emb_model = gensim.downloader.load('glove-twitter-50')
```

3.1 Out-of-Vocabulary (OOV) Handling

To ensure robustness, tokens present in our training set but missing from the GloVe dictionary were identified using the `check_OOV_terms()` function, which computes the set difference between the dataset vocabulary and the GloVe embedding vocabulary:

```
def check_OOV_terms(embedding_model, word_listing):
    embedding_vocab = set(embedding_model.key_to_index.keys())
    oov = set(word_listing).difference(embedding_vocab)
    return list(oov)
```

We recorded 2,743 OOV terms, corresponding to approximately 31.52% of our unique vocabulary. This high OOV rate is consistent with Twitter data, which frequently contains neologisms, intentional misspellings, and domain-specific terms not present in standard corpora.

The `build_embedding_matrix()` function constructed the final embedding matrix:

- For terms present in GloVe: the pre-trained 50-dimensional vectors were loaded directly
- For OOV terms: embeddings initialized via uniform random distribution $U(-0.05, 0.05)$

This mapping was loaded into a static (non-trainable) Keras Embedding layer to preserve the latent spatial relationships learned by GloVe. Tweets were padded to a maximum sequence length of 50 tokens using post-padding with `pad_sequences()`:

```
x_train_padded = pad_sequences(X_train, maxlen=50, padding='post',
                               value=word_to_idx['<PAD>'])
```

4 Model Architectures

We designed two Recurrent Neural Network configurations and one Transformer-based architecture to establish comparative benchmarks. All architectures were implemented using Keras and TensorFlow.

4.1 Baseline Bi-LSTM

The Baseline model architecture processes the static embedding matrix through a series of layers. The complete architecture is defined by the `create_bi_lstm_model()` function with the `double=False` flag:

```
Input (integer token ids)
-> Embedding(vocab_size, 50, trainable=False, mask_zero=True)
-> SpatialDropout1D(p=0.2)
-> Bidirectional(LSTM(256, return_sequences=False, recurrent_dropout=0.1))
-> Dropout(p=0.5)
-> Dense(4, activation='softmax')
```

The `SpatialDropout1D` layer drops entire feature maps (token embeddings) rather than individual elements, promoting feature independence between embedding dimensions. The `bidirectional LSTM` processes the sequence in both forward and backward directions, concatenating both hidden states to capture full contextual dependencies. The final `Dropout (p=0.5)` provides strong regularization before the classification head.

4.2 Stacked Bi-LSTM

The Stacked model extends the baseline by adding depth (`double=True` flag). It consists of two stacked `Bidirectional LSTM` layers:

```
Input (integer token ids)
-> Embedding(vocab_size, 50, trainable=False, mask_zero=True)
-> SpatialDropout1D(p=0.2)
-> Bidirectional(LSTM(256, return_sequences=True, recurrent_dropout=0.1))
-> Bidirectional(LSTM(128, return_sequences=False, recurrent_dropout=0.1))
-> Dropout(p=0.5)
-> Dense(4, activation='softmax')
```

The first LSTM layer outputs the full sequence (`return_sequences=True`), feeding temporal representations to the second LSTM layer (128 units). The added depth aims to model more complex hierarchical semantic structures inherent in identifying intent — for example, separating

Direct from Judgemental statements. Both LSTM layers apply a recurrent dropout of 0.1 to reduce overfitting in the recurrent connections.

4.3 Transformer: Twitter-roBERTa-base

To leverage contextualized dynamic embeddings, we utilized cardiffnlp/twitter-roberta-base-hate, a RoBERTa model pre-trained on approximately 58 million tweets and fine-tuned for hate speech detection. The model was loaded via HuggingFace's AutoModelForSequenceClassification:

```
model = AutoModelForSequenceClassification.from_pretrained(
    'cardiffnlp/twitter-roberta-base-hate',
    num_labels=4,
    id2label=label_encoding_reverse,
    label2id=label_encoding,
    ignore_mismatched_sizes=True
)
```

Instead of manual lemmatization, we utilized the model's native AutoTokenizer, which applies BPE (Byte-Pair Encoding) tokenization. A dedicated preprocessing step normalized the text for the model: mentions were replaced with @user tokens and URLs with http tokens, consistent with the model's pre-training data format. A max length of 120 tokens was applied with truncation.

The key architectural advantages of RoBERTa over LSTM are:

- Attention mechanism: Multi-head self-attention attends to all token positions simultaneously, capturing long-range dependencies without gradient decay
- Contextual embeddings: Each token's representation is conditioned on the entire sequence, unlike static GloVe vectors
- Pre-training: Initialized from weights learned on massive Twitter corpora, providing strong domain-specific priors
- Subword tokenization: BPE handles rare words and neologisms more effectively than word-level vocabularies

5 Training Setup and Evaluation

5.1 Bi-LSTM Training Strategy

To assure robust statistical estimation and limit variance related to random initialization, the LSTM models were evaluated across three distinct random seeds: 42, 111, and 321. Both TensorFlow and NumPy random seeds were set at the start of each run:

```
seeds = [42, 111, 321]
for seed in seeds:
    np.random.seed(seed)
    tf.random.set_seed(seed)
```

Both the Baseline and Stacked models were compiled using the Adam optimizer (Learning Rate = 0.001) and Sparse Categorical Crossentropy loss. Models were trained for 10 epochs with a

batch size of 256. A `ModelCheckpoint` callback monitored the validation Macro F1-score to save the most optimal weights:

```
cp_callback = ModelCheckpoint(
    filepath=checkpoint_path,
    monitor='val_f1',
    mode='max',
    save_best_only=True
)
```

5.2 Metrics

Model performance was evaluated using four metrics, all computed with macro averaging across classes:

- **Macro F1-Score:** Harmonic mean of precision and recall, equally weighting all classes regardless of support — critical for imbalanced data
- **Accuracy:** Fraction of correctly classified samples — can be misleading with class imbalance
- **Macro Precision:** Average precision across all classes
- **Macro Recall:** Average recall across all classes

The `compute_metrics()` function was implemented using `sklearn`:

```
def compute_metrics(output_info):
    predictions, labels = output_info
    predictions = np.argmax(predictions, axis=-1)
    f1 = f1_score(y_pred=predictions, y_true=labels, average='macro')
    acc = accuracy_score(y_pred=predictions, y_true=labels)
    # ... precision, recall similarly
    return {'f1': f1, 'acc': acc, 'precision': precision, 'recall': recall}
```

5.3 Performance Metrics over Seeds

The average performance across all 6 runs (Baseline and Stacked across 3 seeds) on the validation set yielded:

Metric	Mean	Std Dev
Macro F1	0.3378	± 0.0475
Accuracy	0.7278	± 0.0124
Macro Precision	0.4048	± 0.0880
Macro Recall	0.3397	± 0.0368

The best individual performing model was the Stacked Bi-LSTM (Seed 321), achieving a validation F1-score of 0.4071 and Accuracy of 0.7467.

5.4 Test Set Results (All LSTM Models)

Model (Seed)	Macro F1	Accuracy	Precision	Recall
Baseline (42)	0.3729	0.6714	0.3841	0.3731
Stacked (42)	0.2741	0.6607	0.5815	0.2900
Baseline (111)	0.3607	0.6821	0.4646	0.3510
Stacked (111)	0.3739	0.6821	0.4964	0.3642
Baseline (321)	0.3464	0.6679	0.4117	0.3427
Stacked (321)	0.3758	0.6821	0.4514	0.3641

The Stacked Bi-LSTM variants tended to outperform the Baseline, suggesting that additional depth in the recurrent stack is beneficial for capturing the hierarchical nature of intent classification. However, the variance across seeds (particularly for Stacked seed 42, which underperformed markedly) highlights instability in training.

5.5 Ensemble Method

To further optimize predictions, we constructed an ensemble utilizing Soft Voting across the top models. The `ensemble_soft_vote()` function averages predicted class probabilities from selected models:

```
def ensemble_soft_vote(models, X_val):
    all_probs = [model.predict(X_val, verbose=0) for model in models]
    avg_probs = np.mean(all_probs, axis=0)
    final_predictions = np.argmax(avg_probs, axis=1)
    return final_predictions
```

Two ensemble strategies were evaluated:

- Full ensemble (all 6 models): yielded moderate improvement by averaging out individual model noise
- Top-2 model ensemble: selected by sorting models on validation F1 (`np.argsort(f1_scores)[-2:]`) — this yielded a Macro F1-score of 0.3783 and an Accuracy of 0.7400 on validation

The top-2 ensemble demonstrated that selective ensembling outperforms full ensembling, as including weaker models (especially Stacked seed 42) can degrade aggregate performance.

6 Transformer Evaluation

6.1 Preprocessing for Transformer

Unlike the LSTM pipeline, the Transformer preprocessing retained the raw tweet content without stemming or lemmatization. A lightweight normalization step replaced user mentions and URLs with generic tokens, consistent with the model's pre-training format:


```
def preprocess_trans(text):
    tokens = text.split(' ')
    tokens = ['@user' if t.startswith('@') and len(t)>1 else t for t in tokens]
    tokens = ['http' if t.startswith('http') else t for t in tokens]
    return ' '.join(tokens)
```

The datasets were then converted to HuggingFace Dataset objects and tokenized using the model's AutoTokenizer with max_length=120 and truncation enabled. A DataCollatorWithPadding was used during training to dynamically pad batches to their maximum sequence length.

6.2 Fine-Tuning Configuration

The RoBERTa transformer was fine-tuned using the HuggingFace Trainer API. Training arguments were configured as follows:

Hyperparameter	Value
Learning rate	2e-5
Weight decay	0.01
Train batch size	8 per device
Eval batch size	8 per device
Epochs	1
Evaluation strategy	Per epoch
Best model selection	Load best at end

The learning rate of 2e-5 is a standard value for fine-tuning pre-trained transformers, small enough to preserve learned representations while adapting to the new classification task. The model natively processes un-stemmed text, extracting vastly superior contextual representations compared to static GloVe embeddings.

6.3 Test Set Performance

Metric	Score
Macro F1	0.4807
Accuracy	0.7500
Macro Precision	0.5703
Macro Recall	0.4821

The Transformer model achieved an F1-score of 0.4807, marking an absolute improvement of roughly +10% over the best Stacked Bi-LSTM model on the test set. The higher precision (0.5703)

relative to recall (0.4821) suggests the model is conservative — when it predicts a minority class, it is reasonably accurate, but it still misses many minority-class instances.

7 Task 7: Error Analysis

Despite improvements introduced by the Transformer architecture, a deep inspection of the predictions highlighted critical algorithmic blind spots stemming primarily from dataset composition and the inherent ambiguity of intent classification.

7.1 Class Imbalance and Majority Bias

Both the Bi-LSTM and Transformer architectures exhibited a strong bias toward the non-sexist (-) and DIRECT classes, which represent the overwhelming majority of the training data. This skew results in minority classes being absorbed by majority distributions.

The analysis of misclassified samples was performed by iterating over prediction-label pairs:

```
for idx, (pred, label) in enumerate(zip(predicted_labels, y_test)):
    if pred != label:
        print(f'Predicted: {label_encoding_reverse[pred]},
              Actual: {label_encoding_reverse[label]}')
```

A targeted inspection of JUDGEMENTAL and REPORTED misclassifications was also performed by filtering on label == 2 or label == 3, providing direct evidence of the model's struggles with minority classes.

7.2 Failure on 'JUDGEMENTAL' and 'REPORTED' Classes

The models severely struggle to classify JUDGEMENTAL and REPORTED tweets. Semantic boundaries between a user natively expressing a sexist thought (Direct) and quoting/reporting one (Reported) are highly subtle. The following examples highlight key failure patterns:

Example 1 — Implicit Sexist Context:

Actual: REPORTED | Predicted: NON-SEXIST (-)

"@CATTRACHAS and @RFKHumanRights took the case of #VickyHernandez to the @CorteIDH and won..."

Analysis: The tweet reads like a formal news report. The model failed to detect the implicit sexist context of the reported event. The absence of explicit sexist language caused the classifier to default to the majority NON-SEXIST class.

Example 2 — Missing Multi-turn Context:

Actual: JUDGEMENTAL | Predicted: NON-SEXIST (-)

"@ThatUmbrella I can't imagine what the possible beef would have been but you do realize that you two live close enough..."

Analysis: The intention is to judge an overarching situation, but the model lacks the broader multi-turn context required to classify this correctly. The tweet in isolation appears neutral.

Example 3 — Keyword Toxicity Bias:

Actual: REPORTED | Predicted: DIRECT

"@BoopBepp @Helloth07860784 @Bronwen85 Yup I hate when men rape and kill women."

Analysis: The presence of high-toxicity keywords ('rape', 'kill') caused the model to aggressively flag it as DIRECT sexism, rather than recognizing the sarcastic/reported rhetorical structure.

7.3 Transformer vs. LSTM Paradigm

While LSTMs completely failed to isolate the JUDGEMENTAL class (often predicting everything as Direct or Non-Sexist), the Transformer managed to better differentiate the top two majority classes. A key observation from the error analysis:

- LSTM models: Essentially collapsed the JUDGEMENTAL class into DIRECT or NON-SEXIST predictions due to limited contextual understanding from static GloVe embeddings
- Transformer (RoBERTa): The attention mechanism proved more adept at ignoring grammatical noise and capturing semantic nuance, yet without balanced class weights, it still optimized for the macro accuracy driven by the NON-SEXIST (-) class
- Confusion matrices: Both architectures showed a dense off-diagonal pattern concentrated in the NON-SEXIST column, confirming the majority-class absorption effect

7.4 OOV Impact Analysis

The 31.52% OOV rate in the LSTM pipeline had a direct negative impact on representation quality. Twitter-specific tokens (hashtag derivatives, slang, abbreviations) that survived the cleaning pipeline but were absent from GloVe were initialized with random noise vectors. This random initialization means semantically related OOV terms (e.g., different slang for the same concept) may be placed far apart in the embedding space, degrading the classifier's ability to generalize.

The Transformer pipeline bypassed this issue entirely through BPE tokenization, which decomposes unknown words into subword units always present in the model's vocabulary — a significant architectural advantage for noisy social media text.

8 Conclusions and Future Work

This project explored the complexities of multi-class intent classification for sexism detection. While custom Bi-LSTM networks establish a functional baseline, pre-trained Transformers (RoBERTa) fine-tuned on hate-speech domains showcase vastly superior contextual understanding, reaching an F1-score of 0.4807 on the test set. The Transformer delivered approximately +10% absolute improvement over the best LSTM model.

The primary bottleneck remains the severe dataset imbalance, which causes all architectures to under-classify minority categories (JUDGEMENTAL, REPORTED). Future work should address these limitations through the following strategies:

8.1 Data Augmentation

Synthetic oversampling of JUDGEMENTAL and REPORTED classes using back-translation or LLM-based generation could substantially improve minority class representation. Techniques such as:

- Back-translation: Translate minority-class tweets to an intermediate language (e.g., Spanish) and back to English to generate paraphrases with preserved intent
- LLM generation: Prompt a large language model to generate synthetic examples of JUDGEMENTAL and REPORTED tweets that mimic the style and context of real examples
- SMOTE-based approaches: Apply contextual augmentation in the embedding space for LSTM models

8.2 Class Weighting

Modifying the cross-entropy loss function during training to heavily penalize misclassifications on minority classes. This can be implemented in Keras via the `class_weight` parameter in `model.fit()` or by computing inverse-frequency weights:

```
from sklearn.utils.class_weight import compute_class_weight
class_weights = compute_class_weight('balanced',
                                     classes=np.unique(y_train),
                                     y=y_train)
```

8.3 Contextual Meta-features

Injecting features regarding the author's history or conversational thread context to help disambiguate Direct from Reported text. This includes:

- Thread-level context: Including parent tweets in the input to resolve anaphoric references
- Author signal features: Historical posting patterns or profile metadata
- Sarcasm detection: A dedicated sarcasm module could help identify REPORTED and JUDGEMENTAL tweets that use ironic language

8.4 Advanced Architectures

- Larger Transformers: Fine-tuning larger models (RoBERTa-large, DeBERTa) may yield additional performance gains
- Multi-task learning: Jointly training on Task 1 (sexism detection) and Task 2 (intent) may provide complementary supervisory signal
- Focal Loss: Replacing cross-entropy with focal loss dynamically down-weights easy examples, focusing training on hard minority-class samples

References

[1] Rodrigo Agerri et al., "EXIST 2021: sEXism Identification in Social neTworks," IberLEF 2021 Proceedings. Workshop on Iberian Languages Evaluation Forum.

[2] François Chollet et al., Keras deep learning library. <https://keras.io>, 2015.

- [3] Jacob Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL 2019.
- [4] Yinhan Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019.
- [5] Jeffrey Pennington, Richard Socher, Christopher D. Manning, "GloVe: Global Vectors for Word Representation," EMNLP 2014.
- [6] Sepp Hochreiter and Jurgen Schmidhuber, "Long Short-Term Memory," Neural Computation 9(8):1735-1780, 1997.
- [7] Thomas Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," EMNLP 2020 (System Demonstrations).
- [8] Cardiff NLP, "Twitter-roBERTa-base for Hate Speech Detection," HuggingFace Model Hub, <https://huggingface.co/cardiffnlp/twitter-roberta-base-hate>.